

Lec-03

Tuesday, 1 September 2026 4:04 pm

TABLE 2.3 Computed values of $f(x) = \frac{\sqrt{x^2 + 100} - 10}{x^2}$ near $x = 0$

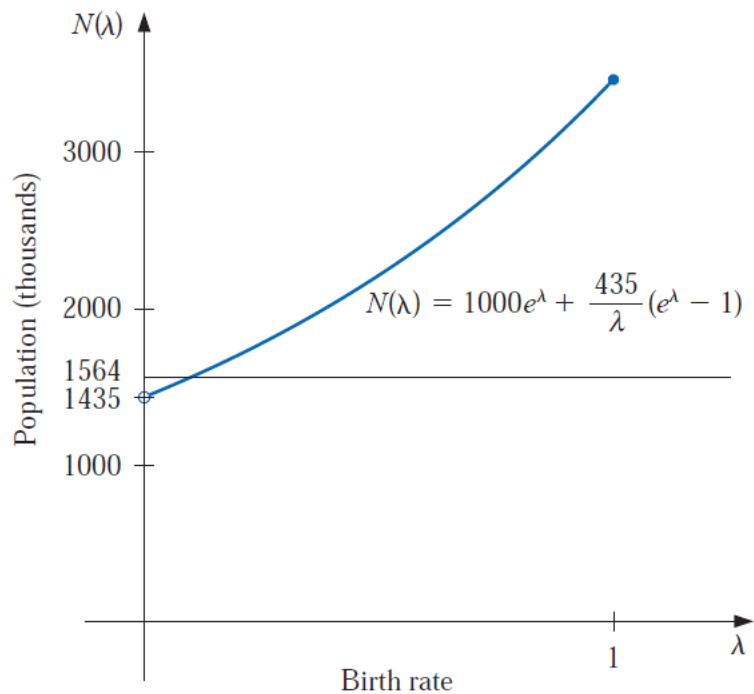
x	$f(x)$
± 1	0.049876
± 0.5	0.049969
± 0.1	0.049999
± 0.01	0.050000
± 0.0005	0.050000
± 0.0001	0.000000
± 0.00001	0.000000
± 0.000001	0.000000

$$\begin{aligned}
 \frac{\sqrt{x^2 + 100} - 10}{x^2} &= \frac{\sqrt{x^2 + 100} - 10}{x^2} \cdot \frac{\sqrt{x^2 + 100} + 10}{\sqrt{x^2 + 100} + 10} \\
 &= \frac{x^2 + 100 - 100}{x^2(\sqrt{x^2 + 100} + 10)} \\
 &= \frac{x^2}{x^2(\sqrt{x^2 + 100} + 10)} \\
 &= \frac{1}{\sqrt{x^2 + 100} + 10}.
 \end{aligned}$$

Solutions of Equations in One Variable

$$\frac{dN(t)}{dt} = \lambda N(t),$$

whose solution is $N(t) = N_0 e^{\lambda t}$, where N_0 denotes the initial population.



$$\frac{dN(t)}{dt} = \lambda N(t) + v,$$

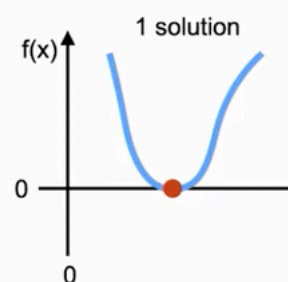
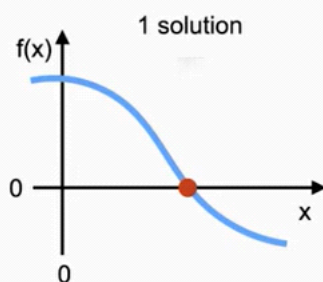
$$N(t) = N_0 e^{\lambda t} + \frac{v}{\lambda} (e^{\lambda t} - 1).$$

Suppose a certain population contains $N(0) = 1,000,000$ individuals initially, that 435,000 individuals immigrate into the community in the first year, and that $N(1) = 1,564,000$ individuals are present at the end of one year. To determine the birth rate of this population, we need to find λ in the equation

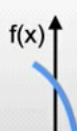
$$1,564,000 = 1,000,000e^\lambda + \frac{435,000}{\lambda}(e^\lambda - 1).$$

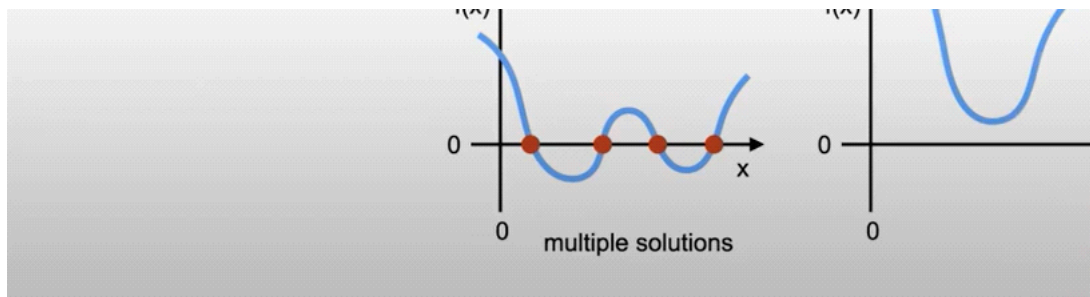
Solving Nonlinear Equations

- Goal: given any $f(x)$, find all x for which $f(x) = 0$
 - common engineering problem
 - also called root finding



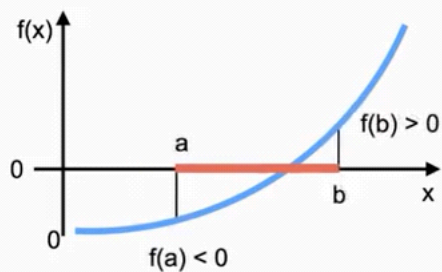
- We could do this graphically:





Bracketing Methods

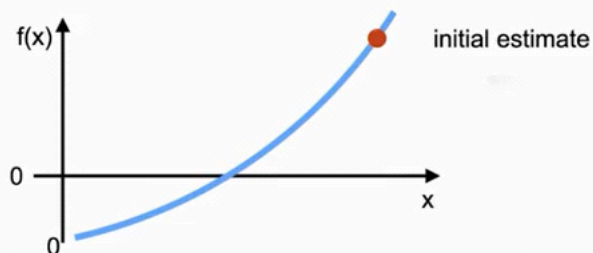
- find an interval $[a, b]$ in which $f(x)$ changes sign



- use a numerical scheme to successively reduce the size of the interval in which $f(x)$ changes sign

Open Methods

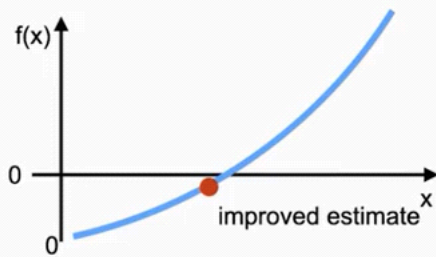
- make an initial guess of the solution, preferably close to the true solution



- use a numerical scheme to improve the estimate

Open Methods

- make an initial guess of the solution, preferably close to the true solution



- use a numerical scheme to improve the estimate
- usually more efficient than bracketing methods
- But: may not yield an answer!

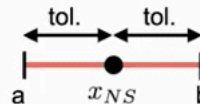
We will cover specific examples for bracketing and open methods in this module

Errors and Tolerances

- Let x_{TS} be the true solution: $f(x_{TS}) = 0$
- Let x_{NS} be the numerical solution: $f(x_{NS}) = \epsilon$ (isn't 0, otherwise $x_{NS} = x_{TS}$, but hopefully $\epsilon \ll 1$)
- **tolerance in solution** maximum difference between true solution and numerical solution

Example: bracketing method:

- set $x_{NS} = \frac{a+b}{2}$ (midpoint of $[a,b]$)



tolerance in solution = $\left| \frac{b-a}{2} \right|$

Comments:

- the tolerance in solution is basically the error band
 - a true solution will be in the interval given by $x_{NS} \pm \text{tolerance in solution}$
- the tolerance in solution can be calculated easily only by bracketing methods

Errors and Tolerances

- Let x_{TS} be the true solution: $f(x_{TS}) = 0$
- Let x_{NS} be the numerical solution: $f(x_{NS}) = \epsilon$

- **tolerance in function**

How far is $f(x_{NS})$ away from $f(x_{TS}) = 0$?

$$|f(x_{TS}) - f(x_{NS})| = |0 - f(x_{NS})| = |f(x_{NS})|$$

$$\text{tolerance in function} = |f(x_{NS})|$$

The Bisection Method

In this chapter we consider one of the most basic problems of numerical approximation, the **root-finding problem**. This process involves finding a **root**, or solution, of an equation of the form $f(x) = 0$, for a given function f . A root of this equation is also called a **zero** of the function f .

The problem of finding an approximation to the root of an equation can be traced back at least to 1700 B.C.E. A cuneiform table in the Yale Babylonian Collection dating from that period gives a sexagesimal (base-60) number equivalent to 1.414222 as an approximation to $\sqrt{2}$, a result that is accurate to within 10^{-5} . This approximation can be found by applying a technique described in Exercise 19 of Section 2.2.

Bisection Technique

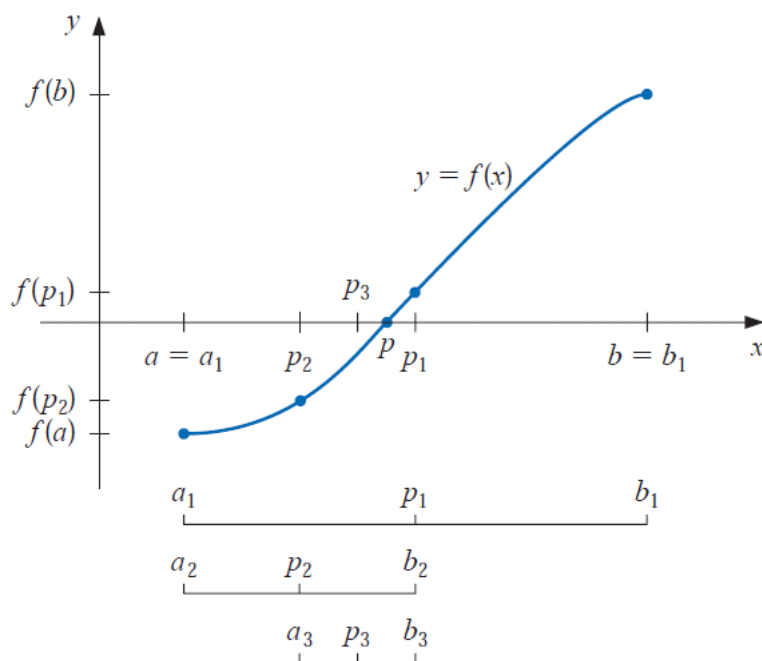
The first technique, based on the Intermediate Value Theorem, is called the **Bisection**, or **Binary-search, method**.

Suppose f is a continuous function defined on the interval $[a, b]$, with $f(a)$ and $f(b)$ of opposite sign. The Intermediate Value Theorem implies that a number p exists in (a, b) with $f(p) = 0$. Although the procedure will work when there is more than one root in the interval (a, b) , we assume for simplicity that the root in this interval is unique. The method calls for a repeated halving (or bisection) of subintervals of $[a, b]$ and, at each step, locating the half containing p .

To begin, set $a_1 = a$ and $b_1 = b$, and let p_1 be the midpoint of $[a, b]$; that is,

$$p_1 = a_1 + \frac{b_1 - a_1}{2} = \frac{a_1 + b_1}{2}.$$

- If $f(p_1) = 0$, then $p = p_1$, and we are done.
- If $f(p_1) \neq 0$, then $f(p_1)$ has the same sign as either $f(a_1)$ or $f(b_1)$.
 - If $f(p_1)$ and $f(a_1)$ have the same sign, $p \in (p_1, b_1)$. Set $a_2 = p_1$ and $b_2 = b_1$.
 - If $f(p_1)$ and $f(a_1)$ have opposite signs, $p \in (a_1, p_1)$. Set $a_2 = a_1$ and $b_2 = p_1$.



Bisection

To find a solution to $f(x) = 0$ given the continuous function f on the interval $[a, b]$, where $f(a)$ and $f(b)$ have opposite signs:

INPUT endpoints a, b ; tolerance TOL ; maximum number of iterations N_0 .

OUTPUT approximate solution p or message of failure.

Step 1 Set $i = 1$;

$FA = f(a)$.

Step 2 While $i \leq N_0$ do Steps 3–6.

Step 3 Set $p = a + (b - a)/2$; (Compute p_i)

$FP = f(p)$.

Step 4 If $FP = 0$ or $(b - a)/2 < TOL$ then

OUTPUT (p); (Procedure completed successfully.)

STOP.

Step 5 Set $i = i + 1$.

Step 6 If $FA \cdot FP > 0$ then set $a = p$; (Compute a_i, b_i .)

$FA = FP$

else set $b = p$. (FA is unchanged.)

Step 7 OUTPUT ('Method failed after N_0 iterations, $N_0 = \cdot, N_0$);

(The procedure was unsuccessful.)

STOP.

Bisection Method

• Procedure:

1) Find an interval $[a, b]$ such that $f(a) \cdot f(b) < 0$

2) Calculate an initial estimate as the midpoint of the interval

$$x_{NS} = \frac{a + b}{2}$$

3) Determine whether true solution is in $[a, x_{NS}]$ or $[x_{NS}, b]$ and update the interval

• if $f(a) \cdot f(x_{NS}) > 0$ then true solution must be in interval $[x_{NS}, b] \Rightarrow$ set $a = x_{NS}$

• if $f(a) \cdot f(x_{NS}) < 0$ then true solution must be in interval $[a, x_{NS}] \Rightarrow$ set $b = x_{NS}$

• if $f(a) \cdot f(x_{NS}) = 0$ then true solution must be x_{NS}

(unless $f(a)$ and $f(b)$ are so small that $f(a) \cdot f(b)$ causes an underflow)

4) Update the solution estimate as the midpoint of the new interval: $x_{NS} = \frac{a + b}{2}$

5) Repeat steps 3-4 until? solution is good enough as given by error measures

tolerance in solution:

$$\left| \frac{b^{(n)} - a^{(n)}}{2} \right| \leq tol_{OK}$$

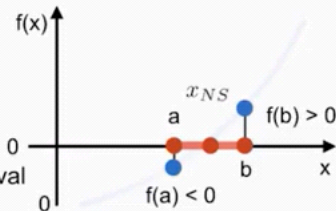
tolerance in function:

$$\left| f(x_{NS}^{(n)}) \right| \leq \epsilon_{OK}$$

estimated relative error:

$$\left| \frac{x_{NS}^{(n)} - x_{NS}^{(n-1)}}{x_{NS}^{(n)}} \right| \leq rel.err_{OK}$$

4



Other stopping procedures can be applied in Step 4 of Algorithm 2.1 or in any of the iterative techniques in this chapter. For example, we can select a tolerance $\varepsilon > 0$ and generate $p_1, \dots, p_N$ until one of the following conditions is met:

$$|p_N - p_{N-1}| < \varepsilon,$$

$$\frac{|p_N - p_{N-1}|}{|p_N|} < \varepsilon, \quad p_N \neq 0, \quad \text{or}$$

$$|f(p_N)| < \varepsilon.$$

Unfortunately, difficulties can arise using any of these stopping criteria. For example, there are sequences $\{p_n\}_{n=0}^{\infty}$ with the property that the differences $p_n - p_{n-1}$ converge to zero while the sequence itself diverges. (See Exercise 17.) It is also possible for $f(p_n)$ to be close to zero while p_n differs significantly from p . (See Exercise 16.) Without additional knowledge about f or p , Inequality (2.2) is the best stopping criterion to apply because it comes closest to testing relative error.

When using a computer to generate approximations, it is good practice to set an upper bound on the number of iterations. This eliminates the possibility of entering an infinite loop, a situation that can arise when the sequence diverges (and also when the program is incorrectly coded). This was done in Step 2 of Algorithm 2.1 where the bound N_0 was set and the procedure terminated if $i > N_0$.

Note that to start the Bisection Algorithm, an interval $[a, b]$ must be found with $f(a) \cdot f(b) < 0$. At each step the length of the interval known to contain a zero of f is reduced by a factor of 2; hence it is advantageous to choose the initial interval $[a, b]$ as small as possible. For example, if $f(x) = 2x^3 - x^2 + x - 1$, we have both

$$f(-4) \cdot f(4) < 0 \quad \text{and} \quad f(0) \cdot f(1) < 0,$$

so the Bisection Algorithm could be used on $[-4, 4]$ or on $[0, 1]$. Starting the Bisection Algorithm on $[0, 1]$ instead of $[-4, 4]$ will reduce by 3 the number of iterations required to achieve a specified accuracy.

The following example illustrates the Bisection Algorithm. The iteration in this example is terminated when a bound for the relative error is less than 0.0001. This is ensured by having

$$\frac{|p - p_n|}{\min\{|a_n|, |b_n|\}} < 10^{-4}.$$

Example 1 Show that $f(x) = x^3 + 4x^2 - 10 = 0$ has a root in $[1, 2]$, and use the Bisection method to determine an approximation to the root that is accurate to at least within 10^{-4} .

Solution Because $f(1) = -5$ and $f(2) = 14$ the Intermediate Value Theorem 1.11 ensures that this continuous function has a root in $[1, 2]$.

For the first iteration of the Bisection method we use the fact that at the midpoint of $[1, 2]$ we have $f(1.5) = 2.375 > 0$. This indicates that we should select the interval $[1, 1.5]$ for our second iteration. Then we find that $f(1.25) = -1.796875$ so our new interval becomes $[1.25, 1.5]$, whose midpoint is 1.375. Continuing in this manner gives the values in Table 2.1. After 13 iterations, $p_{13} = 1.365112305$ approximates the root p with an error

$$|p - p_{13}| < |b_{14} - a_{14}| = |1.365234375 - 1.365112305| = 0.000122070.$$

Since $|a_{14}| < |p|$, we have

$$\frac{|p - p_{13}|}{|p|} < \frac{|b_{14} - a_{14}|}{|a_{14}|} \leq 9.0 \times 10^{-5},$$

Table 2.1

n	a_n	b_n	p_n	$f(p_n)$
1	1.0	2.0	1.5	2.375
2	1.0	1.5	1.25	-1.79687
3	1.25	1.5	1.375	0.16211
4	1.25	1.375	1.3125	-0.84839
5	1.3125	1.375	1.34375	-0.35098
6	1.34375	1.375	1.359375	-0.09641
7	1.359375	1.375	1.3671875	0.03236
8	1.359375	1.3671875	1.36328125	-0.03215
9	1.36328125	1.3671875	1.365234375	0.000072
10	1.36328125	1.365234375	1.364257813	-0.01605
11	1.364257813	1.365234375	1.364746094	-0.00799
12	1.364746094	1.365234375	1.364990235	-0.00396
13	1.364990235	1.365234375	1.365112305	-0.00194

so the approximation is correct to at least within 10^{-4} . The correct value of p to nine decimal places is $p = 1.365230013$. Note that p_9 is closer to p than is the final approximation p_{13} . You might suspect this is true because $|f(p_9)| < |f(p_{13})|$, but we cannot be sure of this unless the true answer is known. ■

The Bisection method, though conceptually clear, has significant drawbacks. It is relatively slow to converge (that is, N may become quite large before $|p - p_N|$ is sufficiently small), and a good intermediate approximation might be inadvertently discarded. However, the method has the important property that it always converges to a solution, and for that reason it is often used as a starter for the more efficient methods we will see later in this chapter.

Theorem 2.1 Suppose that $f \in C[a, b]$ and $f(a) \cdot f(b) < 0$. The Bisection method generates a sequence $\{p_n\}_{n=1}^{\infty}$ approximating a zero p of f with

$$|p_n - p| \leq \frac{b-a}{2^n}, \quad \text{when } n \geq 1. \quad \blacksquare$$

Proof For each $n \geq 1$, we have

$$b_n - a_n = \frac{1}{2^{n-1}}(b-a) \quad \text{and} \quad p \in (a_n, b_n).$$

Since $p_n = \frac{1}{2}(a_n + b_n)$ for all $n \geq 1$, it follows that

$$|p_n - p| \leq \frac{1}{2}(b_n - a_n) = \frac{b-a}{2^n}. \quad \blacksquare \quad \blacksquare \quad \blacksquare$$

Because

$$|p_n - p| \leq (b-a) \frac{1}{2^n},$$

the sequence $\{p_n\}_{n=1}^{\infty}$ converges to p with rate of convergence $O\left(\frac{1}{2^n}\right)$; that is,

$$p_n = p + O\left(\frac{1}{2^n}\right).$$

It is important to realize that Theorem 2.1 gives only a bound for approximation error and that this bound might be quite conservative. For example, this bound applied to the problem in Example 1 ensures only that

$$|p - p_9| \leq \frac{2 - 1}{2^9} \approx 2 \times 10^{-3},$$

but the actual error is much smaller:

$$|p - p_9| = |1.365230013 - 1.365234375| \approx 4.4 \times 10^{-6}.$$

Example 2 Determine the number of iterations necessary to solve $f(x) = x^3 + 4x^2 - 10 = 0$ with accuracy 10^{-3} using $a_1 = 1$ and $b_1 = 2$.

Solution We will use logarithms to find an integer N that satisfies

$$|p_N - p| \leq 2^{-N}(b - a) = 2^{-N} < 10^{-3}.$$

Logarithms to any base would suffice, but we will use base-10 logarithms because the tolerance is given as a power of 10. Since $2^{-N} < 10^{-3}$ implies that $\log_{10} 2^{-N} < \log_{10} 10^{-3} = -3$, we have

$$-N \log_{10} 2 < -3 \quad \text{and} \quad N > \frac{3}{\log_{10} 2} \approx 9.96.$$

Hence, ten iterations will ensure an approximation accurate to within 10^{-3} .

Table 2.1 shows that the value of $p_9 = 1.365234375$ is accurate to within 10^{-4} . Again, it is important to keep in mind that the error analysis gives only a bound for the number of iterations. In many cases this bound is much larger than the actual number required. ■